

BUREAU D'ÉTUDE

Rapport Technique

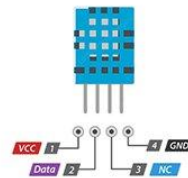
Étude du Capteur de Température et d'Humidité DHT11

Etudiant : Justin AGAMA

Encadrant : Thierry PERISSE



utmel
ELECTRONIC



Composant :	DHT11 — Temperature & Humidity Sensor
Fabricant :	AOSONG Electronics / Distribué par OSEPP
Protocole :	Single-Wire Two-Way (bus série 1 fil)
Alimentation :	3 V à 5,5 V DC
Date du rapport :	Mars 2026

Sommaire

1. Introduction au capteur DHT11
2. Spécifications techniques
3. Caractéristiques électriques
4. Brochage et schéma de câblage
5. Protocole de communication Single-Wire
6. Fonctionnement détaillé de la communication
7. Code de pilotage

1. Introduction au capteur DHT11

Le DHT11 est un capteur numérique de température et d'humidité relative particulièrement répandu dans les applications embarquées et les prototypes électroniques. Développé par AOSONG Electronics, il intègre dans un boîtier compact un capteur résistif d'humidité et une thermistance de type NTC (Negative Temperature Coefficient), couplés à un microcontrôleur 8 bits qui gère l'acquisition, la correction et la transmission numérique des mesures.

La principale caractéristique qui distingue le DHT11 est son interface de communication sur un seul fil bidirectionnel (Single-Wire Two-Way), ce qui simplifie considérablement le câblage : une seule broche de données suffit pour dialoguer avec un microcontrôleur hôte (MCU).

Dans cette étude nous allons écrire une bibliothèque de programmation permettant de piloter le capteur en c.

2. Spécifications techniques

2.1 Humidité relative : Spécifications détaillées

Paramètre	Valeur typique
Résolution	1 %HR
Répétabilité	± 1 %HR
Précision	± 5 %HR
Plage de mesure	30 – 90 %HR
Temps de réponse (1/e)	10 s

2.2 Température : Spécifications détaillées

Paramètre	Valeur typique
Résolution	1 °C
Répétabilité	± 1 °C
Précision	± 1 °C à ± 2 °C
Plage de mesure	0 – 50 °C
Temps de réponse (1/e)	6 – 30 s

3. Caractéristiques électriques

Les caractéristiques électriques ci-dessous sont définies pour VDD = 5 V et T = 25 °C

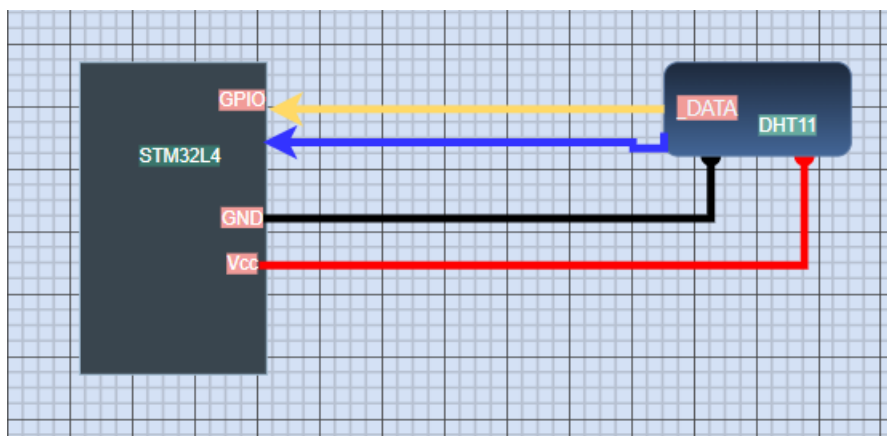
Paramètre	Conditions	Minimum	Maximum
Alimentation (VDD)	DC	3 V	5,5 V
Courant de mesure	Mesure active	0,5 mA	2,5 mA
Courant moyen	Intermittent	0,2 mA	1 mA
Courant de veille	Standby	100 µA	150 µA
Période d'échantillonnage	Minimum	1 s	—

4. Brochage et schéma de câblage

4.1 Description des broches

Broche n°	Nom	Description
1	VCC	Alimentation positive : 3 à 5,5 V DC
2	DATA	Bus de données série bidirectionnel (Single-Wire)
3	NC	Non connectée (broche inutilisée)
4	GND	Masse / référence de tension

4.2 Schéma de câblage typique



5. Protocole de communication Single-Wire

5.1 Présentation générale

Le DHT11 utilise un protocole de communication sur un seul fil de données bidirectionnel (Single-Wire Two-Way). Ce bus est maintenu à l'état haut (HIGH) par la résistance de pull-up en l'absence de communication. Une transaction complète dure environ 4 ms et s'effectue à l'initiative exclusive du MCU hôte : sans signal de démarrage du MCU, le DHT11 reste en mode veille basse consommation et ne transmet rien.

5.2 Format des données transmises

Chaque transaction transmet un mot de 40 bits organisé comme suit, bit de poids fort en premier :

Bits 39–32	Bits 31–24	Bits 23–16	Bits 15–8	Bits 7–0
HR — Partie entière (8 bits)	HR — Partie décimale (8 bits)	T — Partie entière (8 bits)	T — Partie décimale (8 bits)	Somme de contrôle (8 bits)

Vérification de l'intégrité :

La somme de contrôle (checksum) doit être égale aux 8 bits de poids faible de la somme des quatre premiers octets. Si cette condition n'est pas respectée, la trame doit être rejetée et une nouvelle acquisition doit être demandée. Nous nous appuierons sur ce processus dans notre code afin de vérifier que les bites que nous avons stockées est cohérent.

Exemple numérique de vérification du checksum :

Données reçues : 0x28 0x00 0x18 0x00 0x40

HR = 40 % | T = 24 °C | Checksum attendu = (0x28 + 0x00 + 0x18 + 0x00) & 0xFF = 0x40 ✓

5.3 Chronogramme global de communication

La figure ci-dessous illustre la séquence complète d'une transaction entre le MCU (signal hôte) et le DHT11 (signal esclave), depuis le signal de démarrage jusqu'à la transmission de chaque bit de donnée.

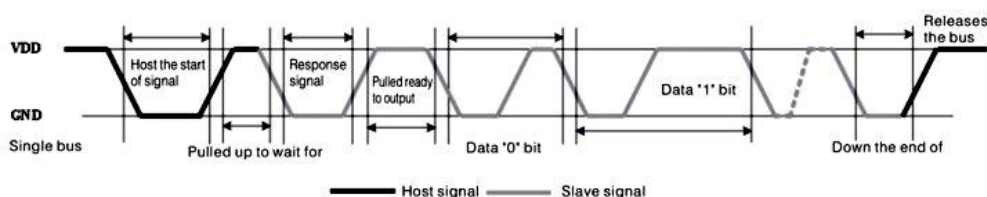


Figure 1 — Chronogramme global de la communication DHT11

6. Fonctionnement détaillé de la communication

6.1 Phase 1 : Signal de démarrage (Start Signal) du MCU

Le MCU initie la communication en forçant la ligne DATA à l'état bas (LOW) pendant une durée minimale de 18 ms. Cette durée est imposée par le DHT11 pour valider un vrai signal de démarrage et ne pas confondre avec du bruit électrique. Le MCU relâche ensuite la ligne (état HIGH) et attend la réponse du capteur pendant 20 à 40 μ s.

Étape	Acteur	Durée
Mise à LOW de la ligne DATA	MCU	≥ 18 ms
Relâchement de la ligne (HIGH)	MCU	20 – 40 μ s

6.2 Phase 2 : Réponse du DHT11

Dès que le DHT11 détecte le front descendant du signal de démarrage, il sort du mode veille et répond selon la séquence suivante :

1. Le DHT11 tire la ligne DATA à l'état BAS pendant 80 μ s (signal de présence / acquittement).
2. Le DHT11 tire la ligne DATA à l'état HAUT pendant 80 μ s (préparation à l'envoi des données).
3. Le DHT11 commence la transmission des 40 bits de données.

Si la ligne reste constamment à l'état HAUT après le signal de démarrage, cela indique que le DHT11 n'a pas répondu : vérifier le câblage, la résistance de pull-up et l'alimentation.

6.3 Phase 3 : Transmission des bits de données

Chaque bit de donnée débute par une impulsion basse de 50 μ s émise par le DHT11. C'est la durée de l'impulsion haute qui suit qui encode la valeur du bit :

Valeur du bit	Impulsion basse (début)	Impulsion haute (encodage)	Interprétation
« 0 »	50 μ s (DHT11)	26 – 28 μ s (DHT11)	Impulsion haute courte → bit 0
« 1 »	50 μ s (DHT11)	70 μ s (DHT11)	Impulsion haute longue → bit 1

Le MCU doit échantillonner la ligne DATA après chaque impulsion basse de 50 μ s pour mesurer la durée de l'impulsion haute. En pratique, on attend un délai d'environ 40 μ s puis on lit l'état de la ligne : si elle est encore à HIGH, le bit vaut « 1 » ; si elle est retombée à LOW, le bit vaut « 0 ».

6.4 Phase 4 : Fin de transmission

Après l'envoi du 40ème bit, le DHT11 tire la ligne DATA à l'état BAS pendant 50 µs, signalant la fin de la transmission. La résistance de pull-up ramène ensuite le bus à l'état HAUT (état de repos). Le DHT11 retourne automatiquement en mode veille basse consommation jusqu'à la prochaine requête du MCU.

7. Code de pilotage

Cette section présente le driver logiciel développé pour piloter le capteur DHT11. Chaque fonction est décrite avec son code source, le raisonnement derrière les choix d'implémentation, et les contraintes temporelles associées au protocole.

7.1 Delay_us() : Génération de délais en microsecondes

```
void Delay_us(uint16_t us)
{
    __HAL_TIM_SET_COUNTER(&htim6, 0);
    while (__HAL_TIM_GET_COUNTER(&htim6) < us);
}
```

La bibliothèque HAL fournit la fonction HAL_Delay(), dont la résolution est limitée à la milliseconde. Or le protocole DHT11 impose des timings à l'échelle de la microseconde (26 µs, 40 µs, 80 µs...). Cette fonction pallie cette limitation en exploitant un timer matériel configuré à 1 MHz, c'est-à-dire 1 tick par microseconde.

Le compteur est d'abord remis à zéro via __HAL_TIM_SET_COUNTER(), puis une boucle d'attente active surveille le compteur jusqu'à ce qu'il atteigne la valeur demandée. L'attente active (busy-wait) est ici délibérée : une approche par interruption introduirait une latence imprévisible, incompatible avec les contraintes temporelles strictes du bus single-wire.

7.2 DHT11_Start() : Envoi du signal de démarrage

```
void DHT11_Start(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* Configuration de la broche en sortie open-drain */
    GPIO_InitStruct.Pin = GPIO_PIN_9;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* Tirer la ligne a 0 pendant 18 ms */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
    HAL_Delay(18);

    /* Relacher la ligne */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET);
    Delay_us(30);

    /* Reconfigurer la broche en entree */
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}
```

Cette fonction implémente la phase d'initialisation du protocole. Son fonctionnement repose sur une reconfiguration dynamique de la broche de données, ce qui est au cœur du principe du bus single-wire bidirectionnel.

Premier point clé : la broche est configurée en mode open-drain et non en push-pull. En mode open-drain, la broche ne peut que forcer la ligne vers le bas (LOW) ou la relâcher (la résistance de pull-up externe ramène alors la ligne à HIGH). Ce mode évite tout court-circuit : si le MCU et le DHT11 actionnent le bus dans des directions opposées simultanément, aucun composant ne peut imposer un niveau HIGH contre un LOW forcé.

La ligne est maintenue à LOW pendant 18 ms. Cette durée est le seuil minimal imposé par la spécification du DHT11 pour distinguer un signal de démarrage valide d'un rebond ou d'un parasite électrique. Après relâchement, la broche est reconfigurée en entrée sans pull interne (GPIO_NOPULL), de sorte que la résistance externe de 5 k Ω seule maintient le bus à l'état haut. Le délai de 30 μ s respecte la fenêtre d'attente de 20 à 40 μ s définie par le protocole avant d'aller lire la réponse du capteur.

7.3 DHT11 CheckResponse() : Vérification de la réponse du capteur

```
uint8_t DHT11_CheckResponse(void)
{
    uint8_t response = 0;

    Delay_us(40);

    if (!HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9))
    {
        Delay_us(80);

        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9))
            response = 1;
    }

    while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9));

    return response;
}
```

Après le signal de démarrage du MCU, le DHT11 répond en deux temps : d'abord 80 μ s à l'état BAS (acquiescement), puis 80 μ s à l'état HAUT (préparation à l'émission). La fonction vérifie successivement ces deux états.

Le délai initial de 40 μ s laisse le temps au DHT11 de prendre le contrôle du bus et de l'abaisser. La première lecture vérifie que la ligne est bien à LOW : si ce n'est pas le cas, cela signifie que le capteur n'a pas répondu cause possible : câblage incorrect, résistance de pull-up absente, ou capteur non alimenté. Dans ce cas, response reste à 0 et la fonction retourne immédiatement.

Si la ligne est bien à LOW, on attend 80 μ s supplémentaires. La ligne doit alors être repassée à HIGH, confirmant que le DHT11 est prêt à envoyer les données.

Le while final est essentiel : sans lui, la transmission des bits commencerait pendant la phase haute de la réponse, décalant la lecture de tous les bits suivants.

Gestion d'erreur : La fonction retourne 0 si le capteur ne répond pas. Ce code de retour est testé dans `DHT11_ReadData()`, qui abandonne l'acquisition en cas d'échec. Les pointeurs de sortie ne sont alors pas modifiés.

7.4 DHT11 ReadBit() : Lecture d'un bit

```
uint8_t DHT11_ReadBit(void)
{
    while (!HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9));

    Delay_us(40);

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9))
    {
        while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9));
        return 1;
    }
    else
    {
        return 0;
    }
}
```

C'est la fonction la plus critique du driver en termes de précision temporelle. Elle décode chaque bit en mesurant la durée de l'impulsion haute émise par le DHT11, selon le principe décrit dans la section 6.3 du présent rapport.

Le `while` initial synchronise la lecture sur le front montant de la ligne : chaque bit commence par 50 µs à LOW, puis la montée vers HIGH démarre l'encodage. Une fois ce front montant détecté, la fonction attend 40 µs avant de lire l'état de la ligne. Ce seuil de 40 µs est le point de discrimination optimal :

- **bit « 0 »** : impulsion haute de 26 à 28 µs. À 40 µs, la ligne est déjà retombée à LOW → lecture : 0.
- **bit « 1 »** : impulsion haute de 70 µs. À 40 µs, la ligne est encore à HIGH → lecture : 1.

Si la ligne est encore à HIGH (bit 1), le second `while` attend que la ligne redescende avant de retourner. Cette étape est indispensable pour aligner le départ de la lecture du bit suivant : sans elle, la lecture s'enchaînerait immédiatement pendant l'impulsion haute encore en cours, corrompant les bits restants.

7.5 DHT11 ReadByte() : Lecture d'un octet

```

uint8_t DHT11_ReadByte(void)
{
    uint8_t i, byte = 0;

    for (i = 0; i < 8; i++)
    {
        byte <= 1;    /* Decalage d'un bit vers la gauche */
        byte |= DHT11_ReadBit();    /* Insertion du bit lu en position 0 */
    }

    return byte;
}

```

Cette fonction implémente un registre à décalage logiciel pour reconstituer un octet à partir de 8 lectures successives de DHT11_ReadBit(). L'ordre de reconstitution respecte le format de trame du DHT11, qui envoie le bit de poids fort (MSB) en premier.

À chaque itération, l'octet en cours est décalé d'un rang vers la gauche (<= 1), libérant ainsi la position 0. Le bit lu via DHT11_ReadBit() est alors inséré en position 0 par un OU logique (|=). Après 8 itérations, le premier bit reçu se retrouve en position 7 (MSB) — exactement comme l'impose le protocole.

Exemple de reconstitution pour la valeur 0b10110000 (0xB0) : Itération 1 : byte = 00000001 (bit MSB = 1). Itération 2 : byte = 00000011 (decalage + bit 0 = 1). ... Itération 8 : byte = 10110000 (0xB0). Le MSB occupe bien la position la plus haute après 7 décalages successifs.

7.6 DHT11 ReadData() : Acquisition complète et vérification

```

void DHT11_ReadData(uint8_t *humidity, uint8_t *temperature)
{
    uint8_t hum_int, hum_dec, temp_int, temp_dec, checksum;

    DHT11_Start();

    if (DHT11_CheckResponse())
    {
        hum_int  = DHT11_ReadByte();
        hum_dec  = DHT11_ReadByte();
        temp_int = DHT11_ReadByte();
        temp_dec = DHT11_ReadByte();
        checksum = DHT11_ReadByte();

        if (checksum == (hum_int + hum_dec + temp_int + temp_dec))
        {
            *humidity  = hum_int;
            *temperature = temp_int;
        }
    }
}

```

Fonction d'entrée principale du driver, `DHT11_ReadData()` orchestre l'ensemble du cycle d'acquisition en appelant successivement les fonctions précédentes. Elle reçoit deux pointeurs vers les variables de l'application qui seront mises à jour en cas de succès complet.

Les 5 octets sont lus dans l'ordre strict imposé par la trame du protocole : humidité entière, humidité décimale, température entière, température décimale, puis somme de contrôle. Les octets décimaux (`hum_dec`, `temp_dec`) sont lus mais non affectés aux variables de sortie. Ce n'est pas un oubli : la résolution du DHT11 étant de 1 °C et 1 %HR, ces octets valent systématiquement 0x00 et ne portent aucune information utile. Ils doivent toutefois être lus pour maintenir la synchronisation sur le bus.

La vérification du checksum est implémentée conformément au protocole : la somme arithmétique des quatre premiers octets doit correspondre au cinquième. En cas d'inégalité, les pointeurs de sortie ne sont pas modifiés, préservant les dernières valeurs valides connues de l'application.

Gestion d'erreur et piste d'amélioration : La fonction comporte deux niveaux de protection implicite : abandon de l'acquisition si le capteur ne répond pas, et non-mise à jour des sorties si le checksum est invalide. Cependant, l'appelant n'a aucun moyen de distinguer un succès d'un échec silencieux, car la fonction ne retourne aucun code d'état. Une amélioration consisterait à modifier le prototype en `uint8_t DHT11_ReadData(uint8_t *humidity, uint8_t *temperature)` et à retourner 1 en cas de succès, 0 en cas d'échec (absence de réponse ou checksum invalide). Cela permettrait à l'application de signaler une erreur à l'utilisateur ou de relancer une acquisition.